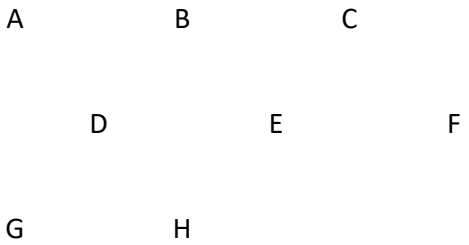
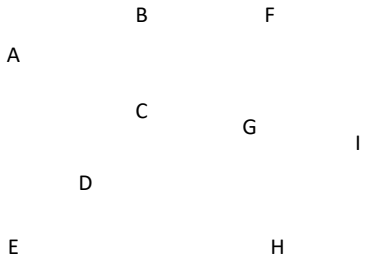
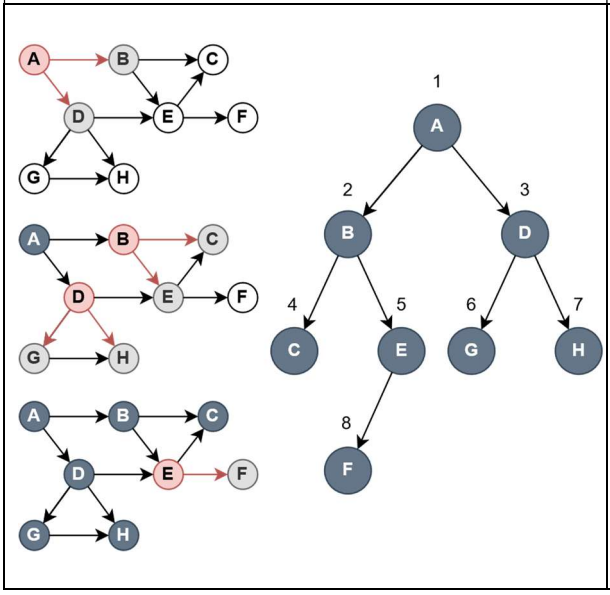
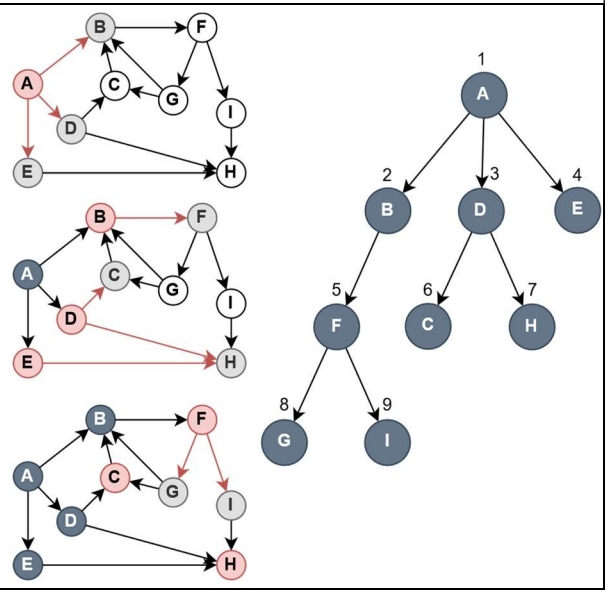
CSE 5004-Graph Theory and Algorithms

Exercise – 2

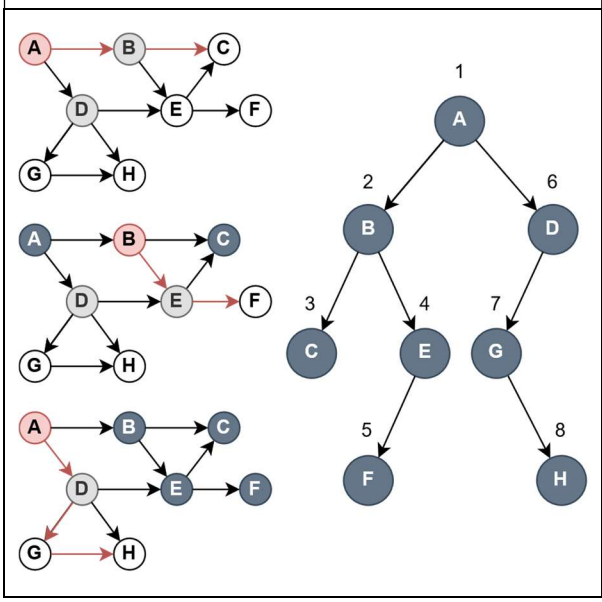
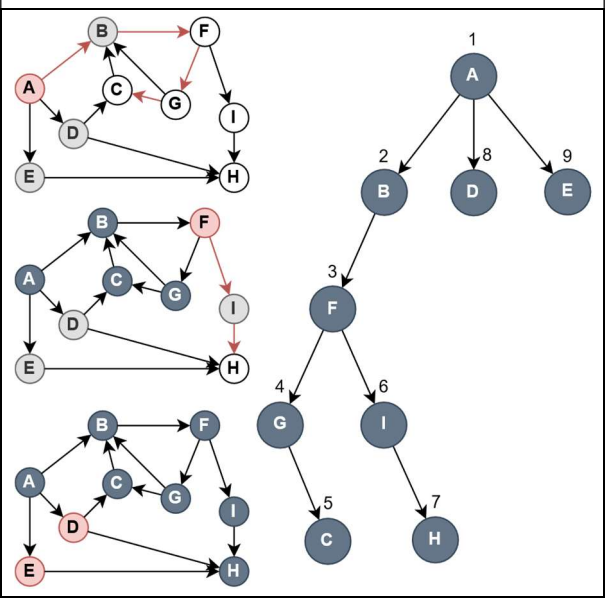
1. Starting from Vertex A, traverse the following graphs with the BFS algorithm. Draw the resulting Breadth-First Trees.





Graph 1 BF Traversal: A B D C E G H F	Graph 2 BF Traversal: A B D E F C H G I
	

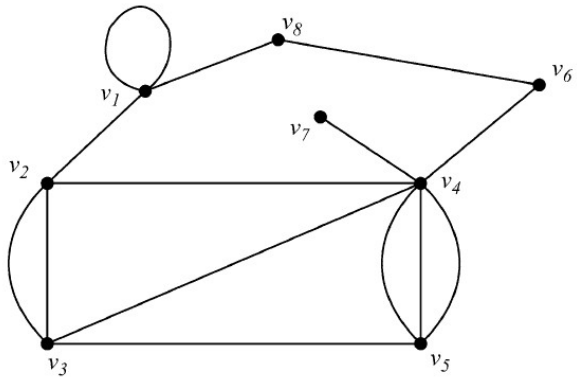
2. Traverse the graphs in Q1 with the DFS algorithm.

Graph 1 DF Traversal: A B C E F D G H	Graph 2 DF Traversal: A B F G C I H D E
	

CSE 5004-Graph Theory and Algorithms

Exercise – 2

3. Modify the following Python code to find the BF traversal of the graph given below. Hint: Discard the self-loops and the parallel edges.



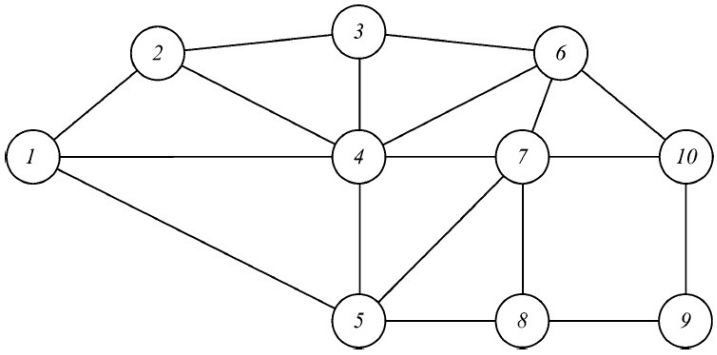

```
# Using a Python dictionary to act as an adjacency list
graph = {
    'v1': ['v2', 'v8'],
    'v2': ['v1', 'v3', 'v4'],
    'v3': ['v2', 'v4', 'v5'],
    'v4': ['v3', 'v5', 'v6', 'v7'],
    'v5': ['v3', 'v4'],
    'v6': ['v4', 'v8'],
    'v7': ['v4'],
    'v8': ['v1', 'v6'],
}
visited = [] # List for visited nodes.
queue = []     #Initialize a queue


def bfs(visited, graph, node): #function for BFS
  visited.append(node)
  queue.append(node)

  while queue:          # Creating loop to visit each node
    m = queue.pop(0)
    print(m, end = " ")

    for neighbour in graph[m]:
      if neighbour not in visited:
        visited.append(neighbour)
        queue.append(neighbour)


# Driver Code
print("Following is the Breadth-First Search")
bfs(visited, graph, 'v5')    # function calling
```

Following is the Breadth-First Search
v5 v3 v4 v2 v6 v7 v1 v8

CSE 5004-Graph Theory and Algorithms

Exercise – 2

4. Modify the following Python code to find the DF traversal of the graph given below.



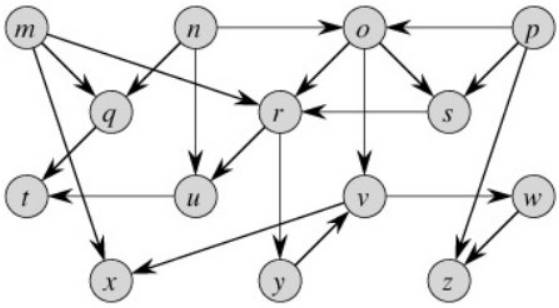

```
graph = {
    1: [2, 4, 5],
    2: [1, 3, 4],
    3: [2, 4, 6],
    4: [1, 2, 3, 5, 6, 7],
    5: [1, 4, 7, 8],
    6: [3, 4, 7, 10],
    7: [4, 5, 6, 8, 10],
    8: [5, 7, 9],
    9: [8, 10],
    10: [6, 7, 9]
}
visited = set() # Set to keep track of visited nodes of graph.

def dfs(visited, graph, node):  #function for dfs
    if node not in visited:
        print (node)
        visited.add(node)
        for neighbour in graph[node]:
            dfs(visited, graph, neighbour)

# Driver Code
print("Following is the Depth-First Search")
dfs(visited, graph, 5)
```

Following is the Depth-First Search
5 1 2 3 4 6 7 8 9 10

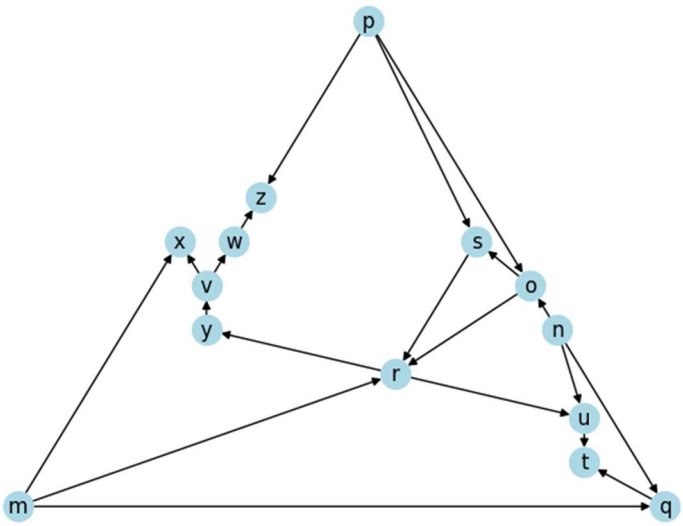
CSE 5004-Graph Theory and Algorithms

Exercise – 2

5. Modify the following Python code to find the topological sort of the graph given below. Paste your final code and output.




```
import networkx as nx
import matplotlib.pyplot as plt

DAG = nx.DiGraph()
edges = [
    ('m', 'q'), ('m', 'r'), ('m', 'x'),
    ('n', 'o'), ('n', 'q'), ('n', 'u'),
    ('o', 'r'), ('o', 's'), ('p', 's'), ('p', 'o'), ('p', 'z'),
    ('q', 't'), ('r', 'u'), ('r', 'y'), ('s', 'r'), ('u', 't'),
    ('v', 'w'), ('v', 'x'), ('w', 'z'), ('y', 'v')
]
for edge in edges:
    DAG.add_edge(*edge)

nx.draw_spring(DAG, with_labels=True)
plt.show()

order = list(nx.topological_sort(DAG))
print(order)
```



['m', 'n', 'p', 'q', 'o', 's', 'r', 'u', 'y', 't', 'v', 'w', 'x', 'z']